INSTITUTO INFNET

ESCOLA SUPERIOR DE TECNOLOGIA

GRADUAÇÃO EM ENGENHARIA DE SOFTWARE



Fundamentos de Desenvolvimento com Java

TP1

Aluno: Ruan Carllo Silva

2 de nov. de 2026

Exercício 1

Comandos utilizados:

```
javac br/edu/infnet/ruan_silva_dr1_tp1/exercicio1/InfoAmbiente.java
java br.edu.infnet.ruan_silva_dr1_tp1.exercicio1.InfoAmbiente
```

Saída obtida:

```
Nome do sistema atual: Mac OS X
Versão do Java: 21.0.13
Diretório de trabalho atual: /Users/ruan/Desktop/Ruan_Silva_DR1_TP1
```

Exercício 2

O mesmo arquivo ".class" funciona em Windows, Linux e macOS porque o compilador "javac" não gera código de máquina nativo, mas sim bytecode, um conjunto de instruções independente de processador e de sistema operacional, empacotado em arquivos ".class". Cada sistema operacional possui sua própria implementação da JVM (“Java Virtual Machine”), que lê esse bytecode e o executa no sistema em uso, traduzindo cada operação para as instruções nativas correspondentes. A diferença entre "javac" e "java" está nisso: o "javac" produz o bytecode portátil, enquanto o "java" o executa por meio da JVM, que faz a ponte entre o formato intermediário e o hardware concreto. Por isso, o mesmo arquivo funciona sem recompilação.

Exercício 3

Sessão "jshell" executada no terminal:

```
ruan@macbook ~/Desktop/Ruan_Silva_DR1_TP1 % jshell
|  Welcome to JShell -- Version 21.0.13
|  For an introduction type: /help intro

jshell> int quantidadeCaixas = 24
quantidadeCaixas ==> 24

jshell> double pesoPorCaixa = 12.5
pesoPorCaixa ==> 12.5

jshell> boolean entregaPrioritaria = true
entregaPrioritaria ==> true

jshell> double pesoTotal = quantidadeCaixas * pesoPorCaixa
pesoTotal ==> 300.0

jshell> double pesoTotalComDesconto = pesoTotal - 15.0
pesoTotalComDesconto ==> 285.0

jshell> pesoTotal > 250.0
$6 ==> true
```

```
jshell> /exit
|  Goodbye
ruan@macbook ~/Desktop/Ruan_Silva_DR1_TP1 %
```

Exercício 4

1) Feito no arquivo java simples.

2) A IA explicou que a classe "ExemploOperadores" é pública e pertence ao package "br.edu.infnet.ruan_silva_dr1_tp1.exercicio4", que serve para organizar e evitar colisão de nomes entre classes. Explicou também que "main" é o ponto de entrada, que "quantidadeCaixas" é um inteiro inicializado com 24 e "pesoPorCaixa" é um double inicializado com 12.5. Sobre o cálculo, disse que "pesoTotal" é obtido por "quantidadeCaixas * pesoPorCaixa", misturando int e double, e que nesse caso Java promove o int para double antes de multiplicar, resultando em 300.0. Por fim, explicou que "cargaPesada" avalia "pesoTotal > 250.0" e gera true, e que os dois "System.out.println" concatenam texto e valores, exibindo "Peso total: 300.0 kg" e "Carga pesada: true".

3) A explicação foi correta em todos os pontos verificáveis, inclusive no detalhe sutil de promoção de tipo, já que misturar int e double numa multiplicação faz o Java converter o int para double automaticamente, e isso ficou claro na resposta. O ponto em que ela simplificou demais foi na descrição do "main", porque a IA chamou de ponto de entrada sem mencionar que sua assinatura específica ("public static void main(String[] args)") é o que a JVM procura na inicialização, e que alterar qualquer um desses quatro elementos (visibilidade, "static", tipo de retorno, parâmetro) impede a execução. Esse é o tipo de detalhe que diferencia uma explicação de superfície de uma explicação completa.

4) Assistentes de código baseados em IA generativa, como o GitHub Copilot, têm sido incorporados diretamente ao IntelliJ IDEA e ao VS Code como plugins nativos, sugerindo trechos de código em tempo real enquanto o desenvolvedor digita. No ecossistema Java especificamente, o framework Spring AI já oferece abstrações para integrar modelos de linguagem (OpenAI, Anthropic, Ollama) a aplicações Spring Boot, permitindo que serviços Java consumam e produzam texto via LLM de forma tipada e idiomática. Além disso, plataformas serverless como AWS Lambda, com o recurso SnapStart, e Google Cloud Run têm ampliado o suporte a runtimes Java, tornando viável usar Java em arquiteturas onde antes a JVM era considerada pesada demais.

Exercício 5

A calculadora de conversão declarou uma variável "double" para a temperatura em Celsius e uma "int" para a distância em quilômetros, aplicando as fórmulas "F = C × 9 / 5 + 32" e "mi = km × 0.621371" diretamente com operadores aritméticos, sem bibliotecas de conversão. A comparação com a temperatura de referência fixa em 100.0 °F foi feita com os operadores relacionais ">", "<" e o "else" para o caso de igualdade, imprimindo o resultado no terminal. O uso de "double" na temperatura é obrigatório porque a fórmula produz casas decimais, enquanto o uso de "int" na distância é suficiente

porque a entrada é inteira e o resultado decimal fica concentrado na variável "double" do lado esquerdo da atribuição.

Exercício 6

O classificador de pedidos leu o peso via "Scanner" e o converteu com "Double.parseDouble" dentro de um bloco "try/catch" para "NumberFormatException", tratando o caminho negativo de texto inválido com mensagem específica antes de qualquer cálculo. Em seguida, tratou o valor negativo numericamente com um "if (peso < 0)" separado, o que dá duas mensagens distintas ao operador, sendo uma para entrada não numérica e outra para número negativo, o que facilita o diagnóstico. A classificação em três categorias usou "if/else" com limites fechados no lado inferior ("<= 10.0" para leve e "<= 50.0" para padrão), garantindo que exatamente 10 kg caia em leve e exatamente 50 kg caia em padrão.

Exercício 7

O classificador foi envolvido em um laço "while (true)" com encerramento por valor sentinela -1.0, e a ordem das validações foi decisiva, porque a comparação com a sentinela vem antes da rejeição de negativos; caso contrário, o -1.0 cairia no ramo de erro e o laço nunca terminaria. O valor sentinela não é contabilizado porque o "break" ocorre antes de qualquer classificação, e cada contador só é incrementado dentro do ramo correspondente. Entradas inválidas e negativos que não sejam a sentinela usam "continue", voltando ao topo do laço sem afetar os contadores, e o total de pedidos é derivado ao final pela soma dos três contadores, sem um quarto contador redundante.

Exercício 8

O menu foi implementado com "do-while", garantindo que ele apareça ao menos uma vez e volte a ser exibido após cada operação até a escolha de sair, e a opção lida foi convertida com "Integer.parseInt" dentro de "try/catch" para "NumberFormatException" antes de entrar no "switch", que possui "default" explícito para opções fora da faixa. A lógica dos exercícios anteriores foi extraída para métodos estáticos reutilizáveis, como "celsiusParaFahrenheit", "quilometrosParaMilhas" e "classificarPedido", sem duplicação de código. As leituras de temperatura, distância e peso foram centralizadas em um único método "lerDoubleNaoNegativo" que retorna "Double" nulo em qualquer falha de validação, e os métodos de execução só seguem adiante quando o retorno não é nulo, o que evita validação espalhada pelo código.

Exercício 9

O programa consolidado reuniu todas as operações em um único executável, com a opção "Sobre" usando "System.getProperty" com a chave "java.version" para exibir a versão do Java em uso e uma linha explicando que a portabilidade decorre do bytecode, que é independente de plataforma, e da JVM de cada sistema operacional, que traduz esse bytecode para código de máquina. A lógica de negócio permaneceu extraída em métodos estáticos, e as validações de entrada continuaram centralizadas no método de leitura, mantendo o mesmo padrão dos exercícios anteriores.

Uso de IA

Para fazer este trabalho, usei o DeepSeek chat como apoio na estruturação do projeto (no caso, as pastas e o pom.xml). No Exercício 4, foi usado para produzir a explicação linha por linha do trecho de código e para pesquisar sobre a incorporação de IA generativa ao ecossistema Java, com exemplos como o GitHub Copilot e o Spring AI. No Exercício 9, foi usado para explicar o comportamento de "NumberFormatException" no "Integer.parseInt" e no "Double.parseDouble", e para conferir os fatores numéricos das fórmulas de conversão de temperatura e distância.